

Contents

Carbon-Copy — Standard Operating Procedure 1

Table of Contents 1

1. What Is Carbon-Copy? 1

2. Prerequisites 2

3. First-Time Setup 3

4. Starting the System 3

5. Accessing the System 4

6. Using the iPhone / iPad App 4

7. Running AI Projects 5

8. Virtual Machines 6

9. Sandbox Runner 8

10. Managing AI Models 9

11. Cloud Storage and File Access 10

12. Homelab Services 11

13. Daily Operations 12

14. Troubleshooting 12

15. Security Guidelines 14

Carbon-Copy — Standard Operating Procedure

Version: 1.0 | Audience: Anyone with access to the repository

Table of Contents

1. What Is Carbon-Copy?

2. Prerequisites

3. First-Time Setup

4. Starting the System

5. Accessing the System

6. Using the iPhone / iPad App

7. Running AI Projects (OpenClaw, NemoClaw, etc.)

8. Creating and Using Virtual Machines (QEMU/KVM)

9. Running Any GitHub AI Repo Safely

10. Managing AI Models

11. Cloud Storage and File Access

12. Homelab Services

13. Daily Operations

14. Troubleshooting

15. Security Guidelines

1. What Is Carbon-Copy?

Carbon-Copy is a self-hosted server platform that runs on your own hardware and gives you:

- **AI Cloud** — Run and use AI services (code analysis, NLP, chat with any model)
- **Virtual Machines** — Full QEMU/KVM VMs you can SSH into, just like a cloud VPS

- **Safe Sandbox** — Run any AI GitHub repo in an isolated container
- **Data Server** — PostgreSQL database + S3-compatible object storage (MinIO)
- **Homelab** — Photo backup, file shares, VPN, ad blocking, file sync
- **Control Panel** — Web app installable on iPhone/iPad like a native app

Everything runs on **your hardware**, behind **your network**. No monthly fees, no cloud dependency.

2. Prerequisites

Minimum Hardware

Resource	Minimum	Recommended
CPU	4 cores (x86_64)	8+ cores with VT-x/AMD-V enabled
RAM	8 GB	16–32 GB
Storage	50 GB free	500 GB+ SSD
OS	Linux (Ubuntu 22.04+)	Ubuntu 22.04 LTS

macOS / Windows: Carbon-Copy runs on Docker Desktop. QEMU/KVM VMs will use software emulation (slower). All other services work fully.

Required Software

Linux / macOS:

```
# Install Docker
curl -fsSL https://get.docker.com | sh           # Linux
# Or: install Docker Desktop from docker.com    # macOS

# Verify
docker --version           # must be 24+
docker compose version     # must be v2.x
```

Windows: 1. Install [Docker Desktop for Windows](#) 2. Enable WSL2 integration in Docker Desktop settings 3. Open **PowerShell as Administrator** for all commands below

Check KVM Support (Linux only — needed for full-speed VMs)

```
grep -c vmx /proc/cpuinfo    # Intel: should be > 0
grep -c svm /proc/cpuinfo    # AMD:   should be > 0
ls /dev/kvm                  # should exist
```

If /dev/kvm doesn't exist:

```
sudo modprobe kvm
sudo modprobe kvm_intel      # or kvm_amd
sudo usermod -aG kvm $USER   # add yourself to kvm group, then log out/in
```

3. First-Time Setup

Step 1 — Get the code

```
git clone https://github.com/Amaanb27241919/Carbon-Copy.git
cd Carbon-Copy
```

Step 2 — Generate secrets

Linux / macOS:

```
bash scripts/generate-secrets.sh
```

Windows (PowerShell):

```
.\scripts\generate-secrets.ps1
```

This creates a `.env` file with randomised passwords and tokens. **Do not skip this step.**

Step 3 — Configure your IP address

Open `.env` and set your server's local IP:

```
HOST_IP=192.168.1.100    # replace with your actual LAN IP
```

Find your IP with:

```
ip route get 1 | awk '{print $7; exit}'    # Linux
ipconfig getifaddr en0                    # macOS
(Get-NetIPAddress -AddressFamily IPv4 | Where-Object { $_.InterfaceAlias -notlike '*Loopback*' })
```

Step 4 — (Optional) Configure AI providers

By default, Carbon-Copy uses **Ollama** (fully local, no API key needed). To use cloud AI:

```
# In .env - add whichever you want:
ANTHROPIC_API_KEY=sk-ant-...    # Claude
OPENAI_API_KEY=sk-...          # OpenAI
HF_API_KEY=hf_...               # HuggingFace
```

Step 5 — (Optional) Homelab extras

```
TAILSCALE_AUTH_KEY=tskey-auth-...    # VPN - get from tailscale.com
DUCKDNS_TOKEN=...                    # Dynamic DNS - get from duckdns.org
DUCKDNS_SUBDOMAIN=mycarbon           # your chosen subdomain
```

4. Starting the System

Core services only

```
bash scripts/start.sh    # Linux/macOS
.\scripts\start.ps1      # Windows
```

Core + Homelab + Storage services

```
bash scripts/start-homelab.sh    # Linux/macOS
.\scripts\start-homelab.ps1      # Windows
```

Stop everything

```
bash scripts/stop.sh
```

Check status

```
docker compose ps
```

All services should show Up with **healthy** after ~60 seconds.

5. Accessing the System

Replace `HOST_IP` with your server's IP address in all URLs below.

What	URL	Notes
PWA Dashboard	<code>http://HOST_IP/app</code>	Install on iPhone via Safari
VS Code	<code>http://HOST_IP/code</code>	Password: <code>CODE_SERVER_PASSWORD</code> from <code>.env</code>
Service Status	<code>http://HOST_IP/status</code>	Uptime Kuma
Photo Backup	<code>http://HOST_IP/photos</code>	Immich (also port 2283)
File Sync	<code>http://HOST_IP/sync</code>	Syncthing
VM Console	<code>http://HOST_IP/console</code>	noVNC browser terminal
API	<code>http://HOST_IP/api</code>	Gateway — requires JWT

Default Login

- **Username:** admin
- **Password:** changeme

Change the admin password immediately after first login.

6. Using the iPhone / iPad App

Install (no App Store needed)

1. Connect your iPhone to the same WiFi network as your server
2. Open **Safari** and go to `http://YOUR_SERVER_IP/app`
3. Tap the **Share** button (box with arrow)
4. Tap **Add to Home Screen**
5. Tap **Add** — Carbon Cloud appears as a full-screen app

Remote access (away from home WiFi)

Set up Tailscale (see Section 12) — then access <http://carbon-copy/app> from anywhere in the world using your iPhone's Tailscale connection.

App Tabs

Tab	What you can do
Dashboard	See if all services are running, recent AI activity
Projects	Start/stop services, run GitHub repos in sandbox
Models	Chat with any AI model, switch providers, pull local models
Files	Browse MinIO storage, upload files
Terminal	View live container logs
Settings	Switch AI provider, enter API keys

7. Running AI Projects

OpenClaw — Code Analysis & Generation

Analyze code:

```
# Get a login token first
TOKEN=$(curl -s -X POST http://HOST_IP/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"changeme"}' | grep -o '"accessToken":"[^"]*"' | cut -d'"' -f4)

# Analyze code
curl -X POST http://HOST_IP/api/openclaw/analyze \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "code": "def fib(n): return fib(n-1) + fib(n-2)",
    "language": "python"
  }'
```

Generate code:

```
curl -X POST http://HOST_IP/api/openclaw/generate \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "Write a rate limiter in Python using Redis",
    "language": "python"
  }'
```

NemoClaw — Language Intelligence

```
# Classify text
curl -X POST http://HOST_IP/api/nemoclave/classify \
```

```

-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"text": "This GPU delivers amazing performance", "labels": ["tech","finance","sports"]}'

# Summarise text
curl -X POST http://HOST_IP/api/nemoclaw/summarize \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"text": "Long article...", "max_length": 100, "style": "bullet-points"}'

# Generate embeddings
curl -X POST http://HOST_IP/api/nemoclaw/embed \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"texts": ["Hello world", "Machine learning is fascinating"]}'

```

Switch AI Model at Any Time

```

# Use Claude for this request
curl -X POST http://HOST_IP/api/chat \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"messages": [{"role": "user", "content": "Explain quantum computing"}], "provider": "claude"}'

# Use local Ollama
curl -X POST http://HOST_IP/api/chat \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"messages": [{"role": "user", "content": "Write a haiku"}], "provider": "ollama", "model": "llama3"}'

```

8. Virtual Machines

Carbon-Copy includes a full QEMU/KVM hypervisor. Each VM is a real virtual machine with its own OS that you can SSH into — exactly like an AWS EC2 instance or a VPS, but running on your own hardware.

Supported Operating Systems

Key	OS	Default RAM	Default Disk
ubuntu-22.04	Ubuntu 22.04 LTS	2 GB	20 GB
ubuntu-24.04	Ubuntu 24.04 LTS	2 GB	20 GB
debian-12	Debian 12 Bookworm	1 GB	20 GB
alpine-3.19	Alpine Linux 3.19	512 MB	8 GB

Step 1 — Create a VM

```
curl -X POST http://HOST_IP/api/kvm/vms \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "name": "my-ai-vm",
  "os": "ubuntu-22.04",
  "ram_mb": 4096,
  "cpus": 4,
  "disk_gb": 50,
  "description": "VM for running OpenClaw experiments"
}'
```

Response:

```
{
  "id": "3f4a1b2c-...",
  "name": "my-ai-vm",
  "os_display": "Ubuntu 22.04 LTS",
  "status": "stopped",
  "ssh_port": 2200,
  "ssh_command": "ssh user@192.168.1.100 -p 2200"
}
```

Step 2 — Boot from installation ISO (first time)

```
curl -X POST http://HOST_IP/api/kvm/vms/VM_ID/start \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"boot_iso": true}'
```

This downloads the OS ISO (~1 GB) and boots into the installer. Use the browser console to complete OS installation.

Open the browser console: Go to http://HOST_IP/console in your browser to see and interact with the VM screen.

Step 3 — Complete OS installation

In the noVNC console: 1. Follow the installer prompts 2. **Important:** When asked about SSH, enable OpenSSH server 3. Set a username and password you'll remember 4. Allow installation to complete and VM to reboot

Step 4 — Boot normally and SSH in

```
# Boot from disk (not ISO) after installation
curl -X POST http://HOST_IP/api/kvm/vms/VM_ID/start \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"boot_iso": false}'
```

```
# Wait ~30 seconds for the OS to boot, then SSH in
ssh your-username@HOST_IP -p 2200
```

Port 2200 is VM slot 0. The second VM uses 2201, third uses 2202, etc.

VM Management Commands

```
# List all VMs
curl http://HOST_IP/api/kvm/vms -H "Authorization: Bearer $TOKEN"

# Stop a VM (graceful)
curl -X POST http://HOST_IP/api/kvm/vms/VM_ID/stop \
  -H "Authorization: Bearer $TOKEN"

# Force stop (like pulling the power)
curl -X POST http://HOST_IP/api/kvm/vms/VM_ID/stop \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"force": true}'

# Delete VM (stops it and removes disk)
curl -X DELETE http://HOST_IP/api/kvm/vms/VM_ID \
  -H "Authorization: Bearer $TOKEN"
```

KVM vs. Software Emulation

	KVM (Linux host)	Software (macOS/Windows)
Speed	Near-native	5–20x slower
Requirement	/dev/kvm + VT-x/AMD-V	None
Status shown in	kvm_enabled: true in API response	kvm_enabled: false

The API response includes `kvm_enabled` so you always know which mode is active.

9. Sandbox Runner

The sandbox safely runs any GitHub AI repository without risking your main system.

Run a GitHub repo

```
curl -X POST http://HOST_IP/api/sandbox/run \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "repoUrl": "https://github.com/username/some-ai-project",
    "name": "my-experiment",
    "cpuLimit": 2,
```

```
"memoryMb": 2048
}'
```

Response (immediate — job runs in background):

```
{
  "runId": "abc123",
  "status": "starting",
  "message": "Cloning repo and building container..."
}
```

Check status and view logs

```
# Poll status
curl http://HOST_IP/api/sandbox/runs/abc123 \
  -H "Authorization: Bearer $TOKEN"

# Get logs
curl "http://HOST_IP/api/sandbox/runs/abc123/logs?tail=50" \
  -H "Authorization: Bearer $TOKEN"
```

Stop a running sandbox

```
curl -X POST http://HOST_IP/api/sandbox/runs/abc123/stop \
  -H "Authorization: Bearer $TOKEN"
```

What the sandbox does automatically

1. Clones the repo (depth 1 — fast)
2. Detects project type: Python → `pip install -r requirements.txt` / Node.js → `npm install` / Go → `go build` / Rust → `cargo build` / Custom Dockerfile → builds it
3. Runs in an isolated container with:
 - No internet access during execution (`--network none`)
 - Hard CPU cap (default: 2 cores)
 - Hard RAM cap (default: 2 GB)
 - Auto-killed after 30 minutes
4. Streams logs back in real time
5. Cleans up everything when done

10. Managing AI Models

Pull a local Ollama model

```
bash scripts/add-model.sh llama3.2      # 2 GB, fast
bash scripts/add-model.sh codellama      # 4 GB, code-focused
bash scripts/add-model.sh mistral        # 4 GB, general purpose
bash scripts/add-model.sh nomic-embed-text # small, for embeddings
```

Windows:

```
.\scripts\add-model.ps1 llama3.2
```

List available models

```
curl http://HOST_IP/api/models \
-H "Authorization: Bearer $TOKEN"
```

Switch default provider

Edit `.env`:

```
DEFAULT_PROVIDER=claude    # or: openai | ollama | huggingface
```

Then restart the model-router service:

```
bash scripts/deploy-project.sh model-router
```

11. Cloud Storage and File Access

Access from iPhone (Files app)

1. Open **Files** app
2. Tap **...** → **Connect to Server**
3. Enter: `smb://HOST_IP`
4. Username: `carbon` | Password: `SAMBA_PASSWORD` from `.env`
5. Shares appear: `shared`, `ai-outputs`, `photos`

Access from Windows

Open File Explorer address bar:

```
\\HOST_IP\shared
```

```
\\HOST_IP\ai-outputs
```

Access from macOS

Finder → Go → Connect to Server → `smb://HOST_IP`

MinIO (S3-compatible API)

```
# Upload a file via API
curl -X POST http://HOST_IP/api/data/files \
-H "Authorization: Bearer $TOKEN" \
-F "file=@/path/to/your/file.pdf" \
-F "bucket=carbon-outputs" \
-F "key=my-folder/file.pdf"

# Download
curl http://HOST_IP/api/data/files/carbon-outputs/my-folder/file.pdf \
-H "Authorization: Bearer $TOKEN" \
-o downloaded-file.pdf
```

Backup

```
bash scripts/backup.sh
```

Creates a compressed PostgreSQL dump + encrypted .env backup in `storage/backups/`. Run nightly via cron:

```
# Add to crontab (Linux/macOS)
crontab -e
# Add this line:
0 2 * * * cd /path/to/Carbon-Copy && bash scripts/backup.sh >> /var/log/carbon-backup.log 2>&1
```

12. Homelab Services

VPN — Access from Anywhere (Tailscale)

1. Create a free account at tailscale.com
2. Generate an auth key: **Settings** → **Keys** → **Generate auth key**
3. Add to .env: `TAILSCALE_AUTH_KEY=tskey-auth-xxxxx`
4. Start homelab services: `bash scripts/start-homelab.sh`
5. Install Tailscale on your iPhone from the App Store
6. Your server appears as `carbon-copy` in your Tailscale network
7. Access everything at `http://carbon-copy/app` from anywhere

Dynamic DNS (DuckDNS)

Gives your home server a stable domain name even when your ISP changes your IP.

1. Create account at duckdns.org
2. Create a subdomain (e.g. mycarbon)
3. Copy your token from the DuckDNS dashboard
4. Add to .env:

```
DUCKDNS_SUBDOMAIN=mycarbon
DUCKDNS_TOKEN=your-token-here
```

5. Your server is reachable at `mycarbon.duckdns.org`

iPhone Photo Backup (Immich)

1. Start homelab: `bash scripts/start-homelab.sh`
2. Install **Immich** from the App Store
3. Open Immich → tap + → Server URL: `http://HOST_IP/photos`
4. Create an account in the Immich web UI (`http://HOST_IP/photos`)
5. Enable **Background App Refresh** in iPhone Settings → Immich
6. Your camera roll backs up automatically over WiFi

Pi-hole (Ad Blocking)

1. Start homelab services
2. Open Pi-hole admin: `http://HOST_IP/dns`

3. Set your router's DNS to HOST_IP (in your router's DHCP settings)
4. All devices on your network now have ad blocking

Syncthing (File Sync)

1. Open Syncthing UI: `http://HOST_IP/sync` (also port 8384)
 2. Install Syncthing on your other devices
 3. Add the server as a remote device using the device ID shown in the UI
 4. Share the `storage/shared` folder — it syncs to all devices automatically
-

13. Daily Operations

Rebuild a single service after code change

```
bash scripts/deploy-project.sh openclaw    # or: nemoclaw, gateway, etc.
```

View live logs

```
docker compose logs -f                # all services
docker compose logs -f openclaw       # one service
docker compose logs -f kvm-manager    # VM manager
```

Database access

```
docker compose exec postgres psql -U carbon -d carbon_db
```

Restart a service

```
docker compose restart openclaw
```

Update to latest code

```
git pull origin main
docker compose up -d --build
```

14. Troubleshooting

Service won't start

```
docker compose ps                # check which services are unhealthy
docker compose logs service-name # read the error
docker compose restart service-name # try a restart
```

VM won't boot / SSH won't connect

```
# Check kvm-manager logs
docker compose logs kvm-manager

# Check if KVM is available on the host
ls -la /dev/kvm

# Check if SSH port is open (e.g., port 2200 for VM slot 0)
nc -zv HOST_IP 2200

# Force stop a stuck VM
curl -X POST http://HOST_IP/api/kvm/vms/VM_ID/stop \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"force": true}'
```

KVM not available (macOS/Windows)

Remove the `devices` block from the `kvm-manager` service in `docker-compose.yml`:

```
# Delete these lines:
devices:
  - /dev/kvm:/dev/kvm
```

VMs will still work — just slower (software emulation).

AI service returns 502

The upstream AI service (`openclaw/nemoclaw`) is down or starting up.

```
docker compose logs openclaw
docker compose restart openclaw
```

Forgot admin password

```
# Reset it directly in the database
docker compose exec postgres psql -U carbon -d carbon_db -c \
  "UPDATE users SET password_hash = '\$2b\$10\$r0FVAcqS6P2LWjTjxPt4guNjJSSLMDEQHgBBqCYrLNP/7vQ9Ly"
# This resets it to: changeme
```

Disk space low

```
# Check usage
df -h
docker system df

# Clean up stopped containers and unused images
docker system prune -f

# List VM disk images
ls -lh /path/to/vm-images/*.qcow2
```

15. Security Guidelines

Before exposing Carbon-Copy to the internet:

1. **Change the default admin password** — log in and change it immediately
2. **Generate secrets** — always run `generate-secrets.sh` before first use, never use the example placeholders
3. **Never commit .env** — it's in `.gitignore` but double-check with `git status`
4. **Use Tailscale instead of port forwarding** — Tailscale is end-to-end encrypted and doesn't expose ports to the internet
5. **Keep VM disk images private** — they may contain sensitive data; treat `storage/` like a personal drive
6. **Rotate INTERNAL_SERVICE_TOKEN regularly** — changing it requires a `docker compose restart`
7. **Backup before updates** — run `bash scripts/backup.sh` before `git pull`

Port exposure summary:

Port	Service	Expose to internet?
80	nginx (all web traffic)	Only via Tailscale or with HTTPS
2200–2209	VM SSH ports	Never — use Tailscale
5900–5909	VM VNC ports	Never
9001	MinIO console	Never
8384	Syncthing	Never
3001	Uptime Kuma	Optional — read-only
11434	Ollama	Never

The safest setup: expose nothing to the internet. Use Tailscale to access everything privately and securely from your iPhone anywhere in the world.